

WebView 与原生交互及替换建议

WebView 与原生交互方式对比及替换建议

一、URL Scheme 交互方式 - 基本统一，只需少量修改

1. 当前实现（UIWebView）

ZYWebBaseViewController 通过 `UIWebViewDelegate` 的 `shouldStartLoadWithRequest` 方法拦截 URL Scheme（如 `contract://`、`login://`），实现原生与 Web 的交互，核心代码如下：

```
- (BOOL)webView:(UIWebView *)webView shouldStartLoadWithRequest:(NSURLRequest *)
request navigationType:(UIWebViewNavigationType)navigationType {
    NSString *urlString = request.URL.absoluteString;

    // 解析URL并处理对应交互逻辑
    if ([urlString containsString:@"contract://"]) {
        // 处理合同相关交互（如打开原生合同详情页）
        return NO; // 阻止WebView默认跳转
    }
    if ([urlString containsString:@"login://"]) {
        // 处理登录相关交互（如跳转原生登录页）
        return NO; // 阻止WebView默认跳转
    }

    return YES; // 允许正常网页跳转
}
```

2. XMWebView 的统一处理逻辑

XMWebView 内部已封装 UIWebView 与 WKWebView 的 URL Scheme 拦截差异，通过统一代理方法回调，无需关注底层 WebView 类型：

WebView 类型	原生拦截方法	统一回调方法
------------	--------	--------

UIWebView	webView: shouldStartLoadWithRequest:	webView: shouldStartLoadWithURL:
WKWebView	webView: decidePolicyForNavigationAction:	webView: shouldStartLoadWithURL:

XMWebView 核心封装代码：

```
// 1. UIWebView的URL拦截与回调
- (BOOL)webView:(UIWebView *)webView shouldStartLoadWithRequest:(NSURLRequest *)
request navigationType:(UIWebViewNavigationType)navigationType {
    if (webView == self.uiWebView) {
        // 触发统一代理回调
        if ([self.delegate respondsToSelector:@selector(webView:shouldStartLoadWithURL:)]) {
            [self.delegate webView:self shouldStartLoadWithURL:request.URL];
        }
        return YES;
    }
    return NO;
}

// 2. WKWebView的URL拦截与回调
- (void)webView:(WKWebView *)webView decidePolicyForNavigationAction:(
WKNavigationAction *)navigationAction decisionHandler:(void (^)(WKNavigationActionPolicy
))decisionHandler {
    if (webView == self.wkWebView) {
        NSURL *URL = navigationAction.request.URL;
        // 调用统一回调逻辑
        [self callback_webViewShouldStartLoadWithRequest:navigationAction.request
navigationType:navigationAction.navigationType];
    }
    decisionHandler(WKNavigationActionPolicyAllow); // 允许正常跳转
}

// 3. 统一回调逻辑（适配两种WebView）
- (BOOL)callback_webViewShouldStartLoadWithRequest:(NSURLRequest *)request
navigationType:(NSInteger)navigationType {
    if ([self.delegate respondsToSelector:@selector(webView:shouldStartLoadWithURL:)]) {
        [self.delegate webView:self shouldStartLoadWithURL:request.URL];
    }
    return YES;
}
```

3. 修改建议

替换为 XMWebView 后，仅需修改代理方法名称，业务逻辑完全复用，代码调整成本低：

```
// 原来的UIWebView代理方法（需废弃）
- (BOOL)webView:(UIWebView *)webView shouldStartLoadWithRequest:(NSURLRequest *)
request navigationType:(UIWebViewNavigationType)navigationType {
    // 原交互逻辑...
}

// 替换后的XMWebView代理方法（新增）
- (void)webView:(XMWebView *)webview shouldStartLoadWithURL:(NSURL *)URL {
    NSString *urlString = URL.absoluteString;

    // 与原逻辑完全一致的交互处理
    if ([urlString containsString:@"contract://"]) {
        // 处理合同相关交互
    }
    if ([urlString containsString:@"login://"]) {
        // 处理登录相关交互
    }
}
```

二、JavaScript 交互方式 - 需要修改代码

1. 当前实现（UIWebView）

ZYWebBaseViewController 使用 UIWebView 的同步方法 `stringByEvaluatingJavaScriptFromString` 与 Web 端 JS 交互（如获取网页标题），核心代码如下：

```
- (void)webViewDidFinishLoading:(UIWebView *)webView {
    [super webViewDidFinishLoading:webView];

    // 同步执行JS，获取网页标题（UIWebView特性）
    NSString *webTitle = [webView stringByEvaluatingJavaScriptFromString:@"document.title"
];
```

```
self.title = webTitle; // 将网页标题设为控制器标题
}
```

2. XMWebView 的问题

XMWebView 未统一 JS 交互 API，UIWebView 与 WKWebView 的方法差异显著，需针对性适配：

特性	UIWebView（同步）	WKWebView（异步）
核心方法	stringByEvaluatingJavaScriptFromString:	evaluateJavaScript:completionHandler:
结果返回方式	直接返回字符串结果	通过 block 回调返回结果（支持多类型）
主线程影响	可能阻塞主线程（耗时 JS 执行时）	不阻塞主线程（异步执行 JS）

3. 修改建议

提供两种适配方案，推荐方案 2 以保持代码整洁性：

方案 1：直接判断 WebView 类型（快速适配，适合简单场景）

在 ZYWebBaseViewController 中根据 XMWebView 实际类型调用对应 JS 方法：

```
- (void)webView:(XMWebView *)webview didFinishLoadingURL:(NSURL *)URL {
    if (webview.wkWebView) {
        // WKWebView：异步执行JS（获取网页标题）
        [webview.wkWebView evaluateJavaScript:@"document.title" completionHandler:^(id _Nonnull result, NSError * _Nullable error) {
            if (result && !error) {
                self.title = result; // 主线程更新UI
            }
        }];
    } else if (webview.uiWebView) {
        // UIWebView：同步执行JS（获取网页标题）
        NSString *webTitle = [webview.uiWebView stringByEvaluatingJavaScriptFromString:@"document.title"];
    }
}
```

```
        self.title = webTitle;
    }
}
```

方案 2：为 XMWebView 扩展统一 JS 交互方法（推荐，符合封装理念）

为 XMWebView 添加分类方法，封装两种 WebView 的 JS 交互差异，对外提供统一接口，降低上层使用成本：

步骤 1：添加方法声明（XMWebView.h 分类）

```
// 新建 XMWebView+JavaScript.h 文件
@interface XMWebView (JavaScript)
/**
 统一执行JS代码的接口
@param javascriptString 待执行的JS代码字符串
@param completionHandler 结果回调（UIWebView同步结果也通过此回调返回）
*/
- (void)evaluateJavaScript:(NSString *)javascriptString
    completionHandler:(void (^ _Nullable)(id _Nullable result, NSError * _Nullable error))
    completionHandler;
@end
```

步骤 2：实现统一方法（XMWebView.m 分类）

```
// 新建 XMWebView+JavaScript.m 文件
@implementation XMWebView (JavaScript)
- (void)evaluateJavaScript:(NSString *)javascriptString
    completionHandler:(void (^ _Nullable)(id _Nullable result, NSError * _Nullable error))
    completionHandler {
    if (self.wkWebView) {
        // WKWebView：直接调用系统异步方法
        [self.wkWebView evaluateJavaScript:javascriptString completionHandler:
        completionHandler];
    } else if (self.uiWebView) {
        // UIWebView：同步执行后通过回调返回结果
        NSString *result = [self.uiWebView stringByEvaluatingJavaScriptFromString:
        javascriptString];
        if (completionHandler) {
```

```
        completionHandler(result, nil); // 无错误，error传nil
    }
}
@end
```

步骤 3：在 ZYWebBaseViewController 中统一使用

```
// 导入分类头文件
#import "XMWebView+JavaScript.h"
- (void)webView:(XMWebView *)webview didFinishLoadingURL:(NSURL *)URL {
    // 统一调用扩展方法，无需判断WebView类型
    [webview evaluateJavaScript:@"document.title" completionHandler:^(id _Nullable result,
    NSError * _Nullable error) {
        if (result && !error) {
            self.title = result; // 更新控制器标题
        }
    }];
}
```

总结

交互方式	替换前后对比	核心修改要点
URL Scheme	逻辑统一，仅代理方法名称变化	将 <code>shouldStartLoadWithRequest</code> 替换为 <code>shouldStartLoadWithURL</code> ，复用原有交互逻辑
JavaScript	接口不统一（同步→异步），需适配两种实现	推荐为 XMWebView 扩展统一方法，上层代码无需感知 WebView 类型，保持整洁性

方案 2（扩展 XMWebView）更符合封装设计理念，后续若完全迁移到 WKWebView，仅需修改分类内部实现，上层业务代码无需调整，可维护性更高。

（注：文档部分内容可能由 AI 生成）